

Lecture 35: Audio-Language Models I

How a model hears

The other ordinary thing you do with a chat model

You record a voice message and send it. It answers the question. It also picks up that you were annoyed, and answers more carefully.

Last lecture was the same problem for pictures. This one is harder, and the reason is worth saying up front.

An image is a grid of pixels. Big, but **finite and static**. Cut it into patches and you are done (L33).

Sound is **16,000 numbers per second, without end**. And the other person is still talking while you answer.

This lecture builds the bridge. The next one makes the model **talk back**.

Two questions that sound the same and are not

1. How does a model hear?

Settled. One recipe, and it is the recipe you already saw last lecture with a different encoder in front of it.

This lecture. By the end you can explain the front end of every audio model shipped in the last three years.

2. How does a model speak?

Not settled. Two competing lineages, and a problem that has no counterpart at all in vision: talking and listening at the same time.

L36. We have to invent a *vocabulary for sound* to get there.

An **audio-language model** takes sound and text as input. The interesting ones do both questions at once, and answer out loud.

Outline

A second of sound is 16,000 numbers

Turning sound into a picture

Bolting ears onto a language model

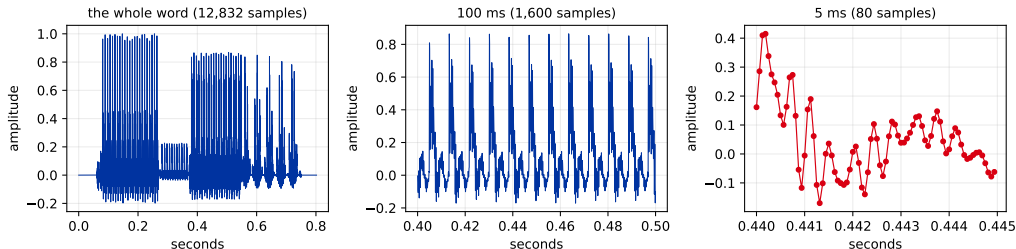
What the cascade throws away

A second of sound is 16,000 numbers

Sound is not a sequence either. It is far worse than a picture.

What a waveform actually is

A microphone measures air pressure. Sampling it 16,000 times a second gives one long list of numbers. Zoom in far enough and you can see the individual samples.



Synthesized speech, not a recording: the word PANIR from ch10, built with a formant model. Real speech is messier - two slides from now you will see exactly how much.

16 kHz is the *speech* convention: nearly all speech energy is below 8 kHz, and sampling must be twice the highest frequency (Nyquist).

It is a convention, not a law. Mimi, EnCodec and MusicGen - *neural audio codecs*, all of which you meet properly in L36 - run at **24** and **32 kHz**. Watch for it when comparing rates.

The problem, as arithmetic

Quantity	Per second
Audio samples at 16 kHz	16,000
English text tokens (for comparison)	≈ 2.5
Ratio	$\approx 6,400\times$

A 30-second clip is **480,000** samples. Attention costs the square of sequence length (ch9), so this is not "expensive" - it is impossible.

Everything in the rest of this lecture is one idea: **get that number down**, without throwing away what makes it speech. Every architecture we look at is a compression scheme wearing a machine-learning hat.

Predict first: could we just feed the raw samples in?

A transformer eats any sequence of vectors. Each audio sample is a number. So why not treat 16,000 samples per second as 16,000 tokens and let attention sort it out?

Predict first: could we just feed the raw samples in?

A transformer eats any sequence of vectors. Each audio sample is a number. So why not treat 16,000 samples per second as 16,000 tokens and let attention sort it out?

Two reasons, and the second is the interesting one.

Cost. Quadratic attention over 480,000 positions for half a minute of audio. No hardware fixes this.

It does not work even when you can afford it. Sample-level models (WaveNet, van den Oord et al., 2016) produce babble that sounds like speech locally and means nothing. Nothing holds structure across 16,000 steps.

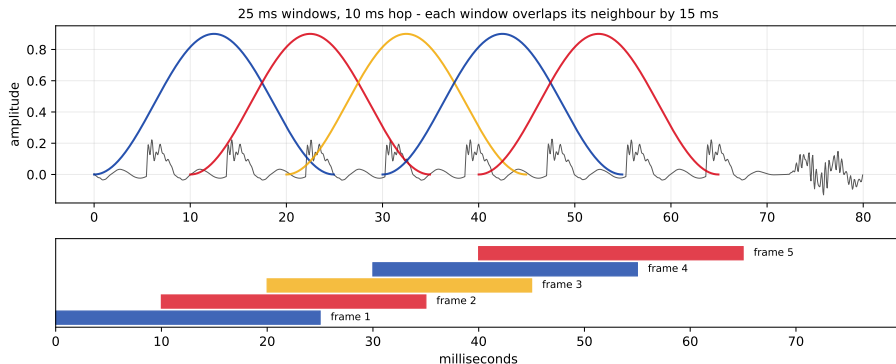
The same lesson as characters versus tokens in text, an order of magnitude more extreme.

Turning sound into a picture

Six slides of signal processing. Every architecture number later in both lectures is arithmetic on what we set here.

Framing: chop it up, and overlap the chops

Take a **25 ms** window, slide it along by a **10 ms** hop, and describe each window separately.



Why 25 ms. Long enough to resolve pitch, short enough that the vocal tract has not moved much. This single choice sets every frame rate downstream.

100 frames per second: a 160× shorter sequence. Not "compression" - each frame is itself a vector of numbers, so the raw *data* barely shrinks. Attention cares about *length*.

One spectrum per frame

Inside a 25 ms window, decompose the signal into the frequencies that make it up. That is the **Fast Fourier Transform (FFT)**; doing it on every frame of a signal is the **Short-Time Fourier Transform (STFT)**.

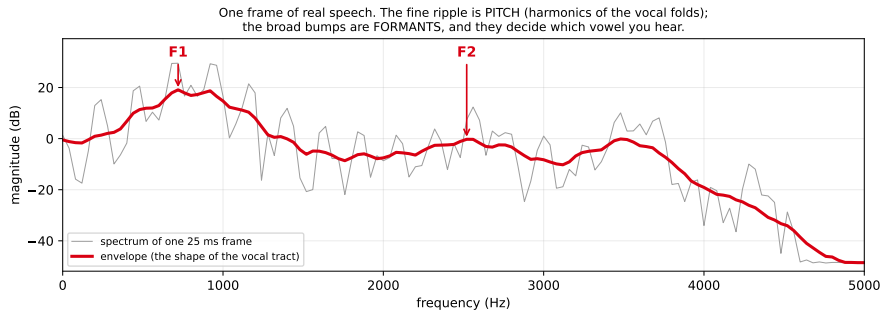
Two things become readable that were invisible in the waveform:

Harmonics - evenly spaced spikes at multiples of the pitch. Their spacing *is* the pitch of the voice.

Formants - broad peaks where the vocal tract resonates. Their positions are what distinguish one vowel from another. This is what makes /a/ an /a/.

A 25 ms window at 16 kHz is 400 samples, which gives **201** frequency bins.

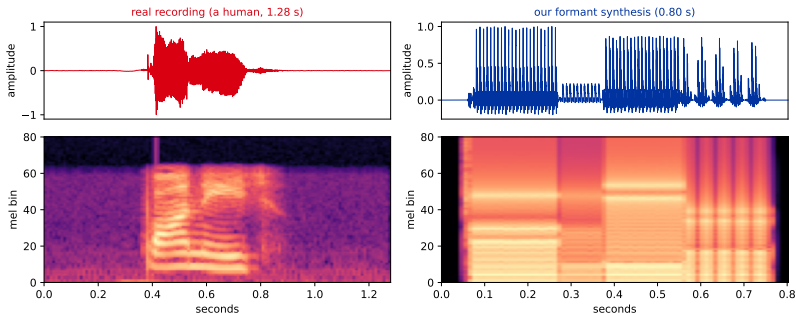
Learning to read one spectrum



Two independent things live in one picture. The **ripple** is the vocal folds buzzing - its spacing is the pitch, and it changes when the same person speaks higher. The **envelope** is the shape of the mouth and throat - its bumps are the formants, and they decide which vowel you hear. **Change the first and the word survives; change the second and it becomes a different word.** That split comes back in L36 with a name: acoustic versus semantic.

Is our synthetic signal honest?

Every figure in this lecture is labelled "synthesized". Here is a **real recording** of the same word beside it, so the label is checkable rather than a disclaimer.



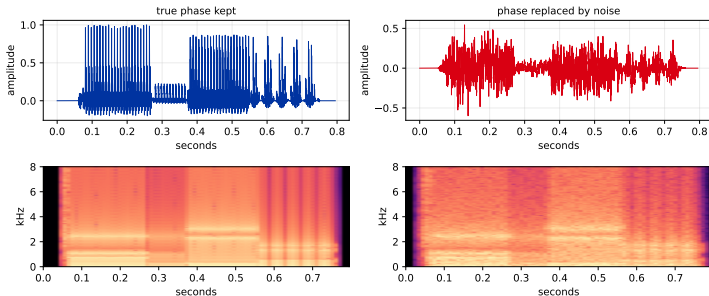
Same word. The real one has noise, breath and drifting pitch; the formants and the burst sit in the same places.

The real one has **breath**, **room noise** and **pitch that drifts** - everything a formant model does not bother to invent. What matches is what this lecture actually claims: the burst, the formant bands, the nasal dip, the trill.

Magnitude and phase, and why we throw one away

The transform returns *complex* numbers: a magnitude (how much of that frequency) and a **phase** (where in its cycle it is). Spectrograms keep only the magnitude.

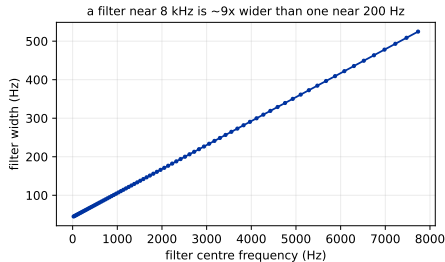
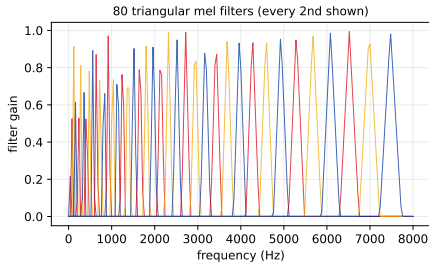
Identical magnitude spectrogram fed in. Only the phase differs.



Remember this slide. A spectrogram *cannot be played back*. That is why speech synthesis needs a **vocoder** - a network whose whole job is to invent plausible phase and turn a spectrogram back into a waveform - and why L36's codecs avoid the problem by working on **raw waveforms** instead.

The mel scale: spend detail where the ear has it

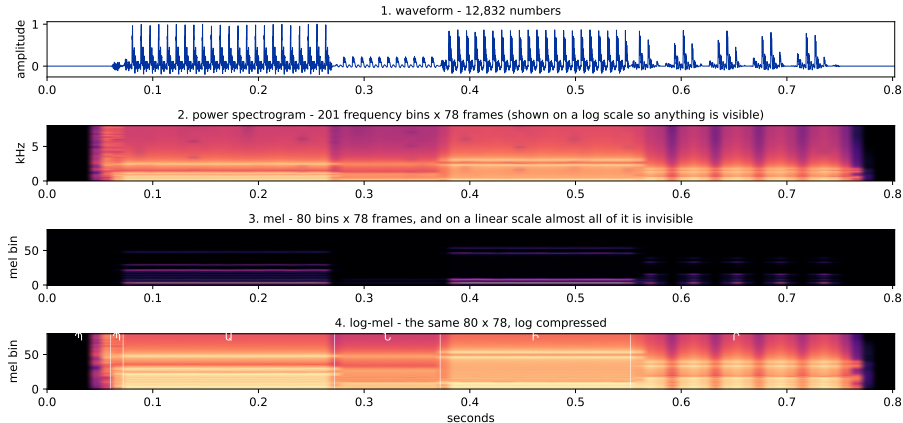
Hearing is roughly logarithmic. The gap from 200 to 400 Hz is enormous; the gap from 7,000 to 7,200 Hz is nearly inaudible. So we pool the 201 bins into **80** bands, narrow at the bottom and wide at the top.



The mapping is $m = 2595 \log_{10}(1 + f/700)$, fitted to human pitch judgements in the 1930s and still in every speech system today.

Now the arithmetic promised three slides ago. 100 frames a second, 80 numbers each, is **8,000 numbers** per second against 16,000 raw samples. The *sequence* got 160× shorter; the *data* only halved. That gap is the whole trick.

Take the log, and you are done



Panel 3 is not a broken figure. On a **linear** scale almost all the structure is invisible, because loudness spans orders of magnitude. The log in panel 4 is what makes it readable - to us, and to a network. **Panel 4 is exactly what Whisper is fed.**

Reading the picture

The spectrogram is not a coloured rectangle. Every region of it says something, and you can now name all of it:

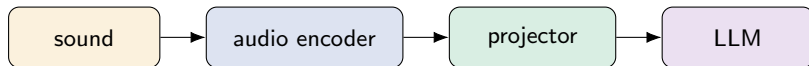
Sound	What to look for	Why
P (closure)	a silent gap	the mouth is shut; a stop consonant is partly <i>silence</i>
P (burst)	a brief wideband smear	the pressure releases all at once
A	two low bands, far apart	first and second formants of an open vowel
N	energy collapses to the bottom	sound is coming out of the nose, not the mouth
I	a low band and a very high one	the tongue is forward; the formants split apart
R	regular vertical stripes	the tongue is tapping, about 25 times a second

Speech recognition was hand-built on features like these for forty years before anyone learned them.

Bolting ears onto a language model

You have seen this recipe before. It was last lecture.

The recipe, which is the same recipe



Replace "audio encoder" with "vision encoder" and this is L33 exactly. The language model is usually **not modified at all**: the projector's whole job is to land audio vectors in the space the text embeddings already live in.

This is the cheapest idea in the chapter and the most important. **Any** modality that can be turned into a sequence of vectors plugs into the same socket. Audio is not special; it is just harder to compress.

Whisper, the encoder almost everyone uses

Whisper (Radford et al., 2022) was trained for speech recognition on 680,000 hours. Almost nobody uses the whole thing - they cut off the encoder and throw the decoder away.

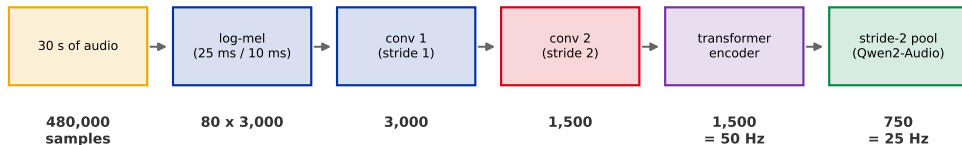
Stage	Size for a 30 s chunk
Raw samples at 16 kHz	480,000
Log-mel frames (10 ms hop)	$80 \times 3,000$
After two convolutions, the second with stride 2	1,500 vectors
Frame rate reaching the transformer	50 Hz

80 mel bins for every size except large-v3, which uses 128. They are not interchangeable - feeding 80 to a 128-bin model fails outright.

Only *one* of the two convolutions has stride 2. Two would halve it twice and give 750, and the number everyone quotes is 1,500.

The whole front end, in one line

Only ONE of the two convolutions has stride 2. Two would give 750, and everyone quotes 1,500.



The transcript of the same 30 seconds is about 75 text tokens.

Every number here came from a decision made earlier in this lecture. The 3,000 is the 10 ms hop. The 80 is the mel filterbank. The 1,500 is one stride-2 convolution. The 750 is a pooling layer somebody added to halve the bill. **Nothing is arbitrary, and nothing is magic.**

Where "semantic" audio features come from

Three models get named constantly in this field, and all three are the same idea you already know from text: **mask part of the input and predict it**, with no labels.

Model	Trained on	Used for
WavLM (Chen et al., 2022)	speech	frames that encode <i>what was said</i>
w2v-BERT (Chung et al., 2021)	speech	the semantic tokens in AudioLM (L36)
Bidirectional Encoder representation from Audio Transformers (BEATs, Chen et al., 2023)	general audio	everything that is <i>not</i> speech: music, dog barks, doors

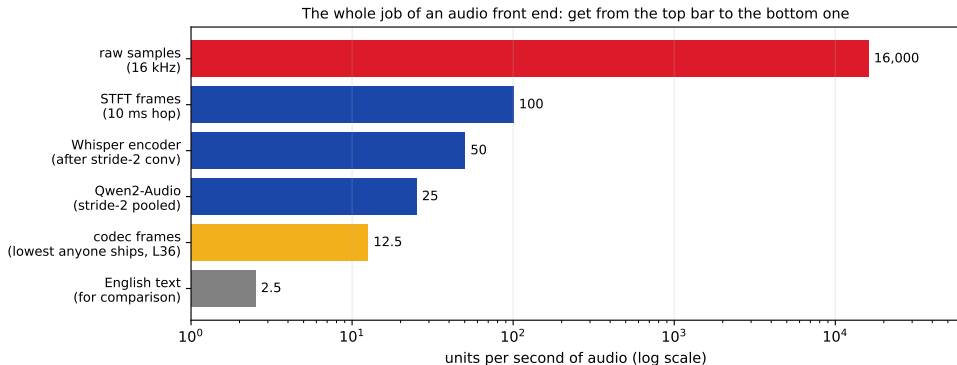
Run k-means on their continuous output frames and you get discrete **units** - a crude alphabet for speech, learned without a single transcript. Hold on to this: it is what makes "semantic tokens" mean something in L36 rather than sounding like magic.

By hand: what 30 seconds costs

Step	Count	Why
audio samples	480,000	$30 \times 16,000$
log-mel frames	3,000	10 ms hop, so 100 per second
after the stride-2 convolution	1,500	50 per second
after a further stride-2 pool	750	25 per second (Qwen2-Audio)
the <i>transcript</i> of the same 30 s	≈ 75	about 2.5 text tokens per second

Even after compressing by a factor of 640, audio still costs about **ten times its own transcript**. Hold on to that ratio - it decides the architecture of every product in this space, and we will meet it again on the bill.

Every rate in this chapter, on one axis

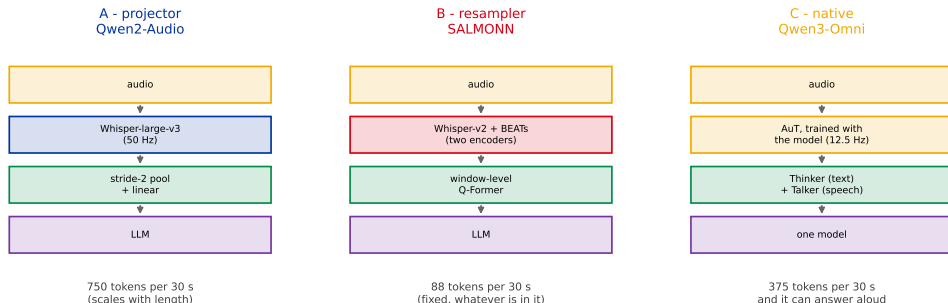


Careful reading these against each other: the top four come from 16 kHz speech, while Mimi runs at 24 kHz. The rates are still comparable because they count *frames per second of audio*, not per sample.

The three answers, side by side

Everyone agrees on "encoder, then language model". The disagreement is **how much of the audio survives the trip** - the same argument as L33, for a different sense.

The same question as the vision chapter: how many tokens does the language model see, and who decides?



Watch the bottom row. It is the only place these three genuinely disagree.

Design A: a projector (Qwen2-Audio)

Qwen2-Audio (Chu et al., 2024) does the simplest thing that works.

- ▶ Whisper-large-v3 encoder, so 128 mel bins in and 50 Hz out.
- ▶ A pooling layer with stride 2, giving **25 Hz** - one vector per 40 ms of audio.
- ▶ A **single linear layer** maps each vector into the language model's embedding space. The audio vectors are then simply placed in the sequence alongside text.

This is LLaVA (L33) with a microphone. Note it is the *single-linear* version, not the two-layer projector of LLaVA-1.5 - the audio encoder is already doing the heavy lifting.

Cost: 30 seconds of audio occupies 750 positions in the context window.

Design B: a resampler (SALMONN)

SALMONN (Tang et al., 2024) refuses to let cost scale with duration.

- ▶ **Two** encoders, concatenated: Whisper-large-v2 for speech, BEATs for everything else.
- ▶ A window-level Q-Former (the resampler idea from L33) compresses each window to a fixed number of learned query vectors.
- ▶ 30 seconds becomes **88 tokens**. One token per **340 ms**, no matter what is in it.

88 against 750. An $8.5\times$ saving, and the budget is now predictable.

And a fixed budget is also a ceiling. Whatever a busy 30 seconds contained, it is getting 88 tokens.

Design C: built in from the start (Qwen3-Omni)

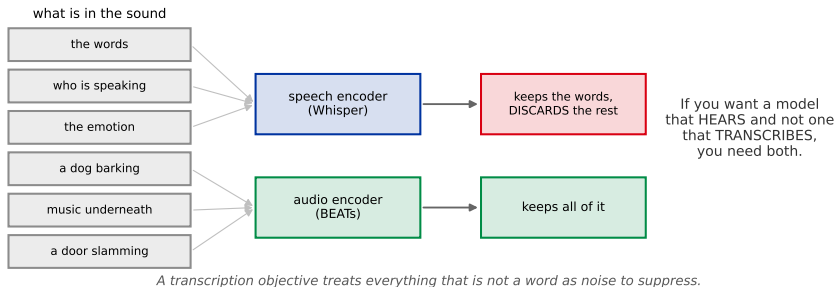
Qwen3-Omni (Qwen Team, 2025) does not bolt an encoder on. The audio encoder is trained with the model.

- ▶ **Audio Transformer (AuT)**, trained from scratch on **20 million hours**, emitting **12.5 Hz** - the lowest rate on the ladder.
- ▶ A **Thinker** that reasons in text and a **Talker** that emits speech, both mixture-of-experts.
- ▶ Theoretical first packet of audio out: **234 ms**.

The same trajectory as vision in L33: bolted-on projector first, trained-together later, as soon as there was enough data to justify it. The Talker half is L36's subject.

Why SALMONN carries two encoders

This looks like redundancy. It is not, and the reason is a genuinely useful thing to know.



Most of the gap between a "speech model" and an "audio model" on benchmarks is this one design choice.

Why that happens, stated plainly

A speech encoder is trained to produce **transcripts**. Everything that is not a word is, to that training objective, **noise to be suppressed**: the dog barking, the door, the music under the dialogue, whether the speaker is a child, whether there are two of them. Whisper is very good at ignoring exactly what you might want to ask about.

So if you want a model that hears, and not merely one that transcribes, you need an encoder that was never taught to throw sound away.

This is most of the gap between "speech model" and "audio model" on benchmarks.

What the cascade throws away

There is an obvious pipeline that avoids all of this. It is worth knowing exactly what it costs, and exactly what it is still better at.

The cascade, and what falls out of it



Both arrows in the middle are **plain text**. Everything the sound carried that text cannot hold dies there:

who is speaking emotion sarcasm
hesitation pace emphasis
two people at once background sound
accent whether it was speech at all

"Say that again, but angrier." There is no answer to this from a transcript. The information was destroyed at the first arrow.

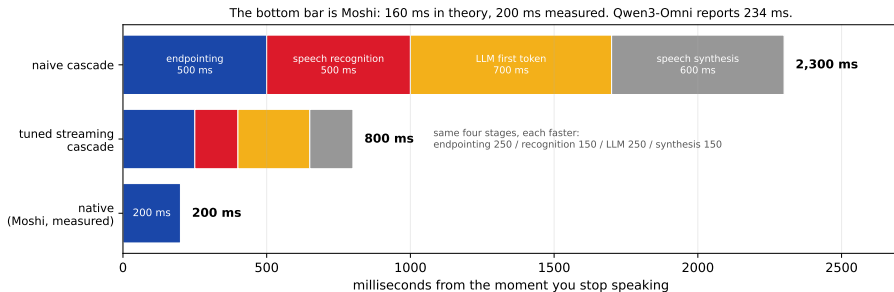
And turn-taking is a guess: a **voice-activity detector (VAD)** decides you have stopped talking. That is precisely why interrupting these systems feels wrong.

Predict first: what does the cascade cost in time?

Four models in a row, each waiting for the last to finish. Guess the delay between you stopping and the first sound coming back.

Predict first: what does the cascade cost in time?

Four models in a row, each waiting for the last to finish. Guess the delay between you stopping and the first sound coming back.



Three tiers, not two - a tuned streaming cascade is much better than a naive one, and saying otherwise would be selling you something. Human conversational turn gaps average about **200 ms**, which is the target the bottom bar is aiming at.

The honest counter-argument

Native models win on latency and on everything text cannot carry. Cascades still run most production systems in 2026, and the reasons are good ones.

For the cascade

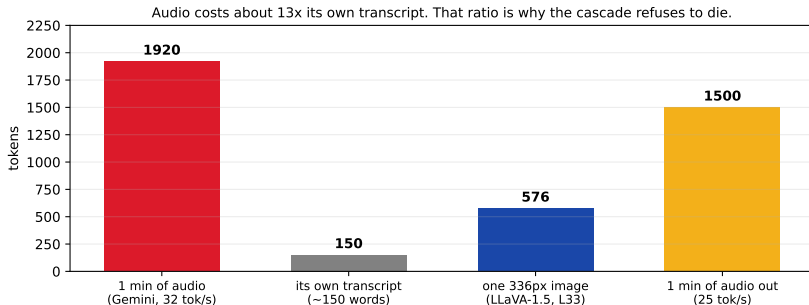
You can read the transcript, so you can debug it, log it, and satisfy an auditor. You can swap any of the three parts for a better or cheaper one. Dozens of vendors compete at each stage.

For native

About a **quarter** of the latency of a tuned cascade, and a tenth of a naive one. Keeps prosody and emotion. Handles overlap and interruption. But it is a black box from a small number of vendors, and you cannot inspect what it "heard".

"The better architecture" is not the same question as "the better system to deploy on Tuesday". Both answers are defensible right now.

The bill



Gemini bills **32 tokens per second** of audio input, so one minute is **1,920** tokens and the documented ceiling is 9.5 hours in a single prompt. Output audio runs about 25 tokens per second.

This is the ratio from the by-hand slide, arriving from a different direction. There we counted *encoder frames* and got about 10 $\times$; here we are reading a *vendor's price list* and get about 13 $\times$. Different systems, different measurements, same order of magnitude - which is what makes it a fact about the modality rather than about one company. Same lesson as L33's 576-tokens-per-image slide: **the modality is the cost.**

Where audio models still fail

Known weak spots

- ▶ Long recordings - the context bill above
- ▶ Counting and separating speakers
- ▶ Exact timestamps ("when was this said")
- ▶ Music, beyond naming the genre
- ▶ Low-resource languages, **Armenian among them**

Whisper hallucinates on silence. Koenecke et al. (2024) found invented phrases in about **1%** of transcriptions, and **38% of those** contained explicit harms - invented violence, made-up names, fabricated authority. They cluster on long non-vocal stretches.

The failure mode is worth pausing on. A recognition model must emit *something*, and trained on 680,000 hours of mostly-speech it has never seen a long silence that meant nothing. The same language prior that makes it fluent makes it confabulate - exactly the vision-model hallucination story from L33, with a different sense organ.

Recap

- ▶ Sound arrives as **16,000 numbers a second**, about 6,400 times the rate of text. Every design in this lecture exists to bring that number down.
- ▶ **25 ms windows, 10 ms hop, 80 mel bins, log**. Six slides of signal processing that turn sound into a picture, and set every frame rate that follows.
- ▶ Keeping only magnitude **throws phase away**, so a spectrogram cannot be played back. Remember it in L36.
- ▶ **Encoder, projector, language model** - the same three boxes as L33. Whisper at 50 Hz, pooled to 25, or resampled to a fixed 88 tokens per 30 s.
- ▶ The **cascade** still wins on debuggability and vendor choice; it costs between 0.6 and 2 seconds of extra latency, and everything a transcript cannot hold.

Next: the model has to answer *out loud*. A language model emits discrete tokens from a fixed vocabulary, and sound is continuous - so we go and invent a vocabulary for sound, then teach the model to talk and listen at the same time.